

Python Grundlagen – Notizen

Kontrollstrukturen, Schleifen und Fehlerbehandlung mit kurzen Gedanken zu jedem Beispiel

Der Code wurde von mir selbst programmiert und von der KI als PDF erstellt.

1. if-Anweisung

```
alter = 18

if alter >= 18:
    print("Du bist volljährig.")
```

Der einfachste Fall: Eine Bedingung wird geprüft, und nur wenn sie True ist, wird der eingerückte Block ausgeführt. Wichtig war mir hier, dass die Einrückung in Python nicht nur Formatierung ist, sondern den Codeblock überhaupt erst definiert.

2. if / else

```
alter = 16

if alter >= 18:
    print("volljährig")
else:
    print("Minderjährig")
```

Hier gibt es für jeden möglichen Fall (True oder False) genau einen Zweig. Ich habe mir gemerkt: else braucht keine eigene Bedingung, weil es automatisch "alles andere" abdeckt.

3. if / elif / else

```
note = 2

if note == 1:
    print("Sehr gut")
elif note == 2:
    print("Gut")
elif note == 3:
    print("Befriedigend")
elif note == 4:
    print("Genügend")
else:
    print("Nicht genügend")
```

Das ist praktisch für mehrere mögliche Fälle, z.B. Notenstufen. Sobald eine Bedingung zutrifft, werden die restlichen elif/else-Zweige übersprungen – das spart im Vergleich zu vielen einzelnen if-Blöcken Rechenzeit und ist übersichtlicher.

4. for-Schleife

```
for i in range(5):
    print(i)
```

range(5) erzeugt die Zahlen 0 bis 4. Am Anfang war mir nicht klar, dass die Endzahl nicht mitgezählt wird – das war ein klassischer Stolperstein für mich.

5. while-Schleife

```
x = 0

while x < 5:
    print(x)
    x = x + 1
```

Im Unterschied zur for-Schleife läuft die while-Schleife, solange eine Bedingung erfüllt ist – die Anzahl der Durchläufe steht also nicht von vornherein fest. Wichtig: x muss innerhalb der Schleife verändert werden, sonst entsteht eine Endlosschleife.

6. break

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

break bricht die Schleife sofort ab, sobald die Bedingung eintritt (hier bei `i == 5`). Deshalb werden nur 0 bis 4 ausgegeben, nicht bis 9. Nützlich, wenn man z.B. bei einem Treffer die Suche abbrechen will, statt die ganze Schleife durchlaufen zu lassen.

7. pass

```
if True:
    pass
```

pass ist nur ein Platzhalter, der nichts tut. Ich nutze das, wenn ich eine Struktur (z.B. eine Funktion oder ein if) schon anlegen will, den Inhalt aber erst später schreibe – ohne pass würde Python einen Fehler wegen des fehlenden Blocks werfen.

8. try / except

```
try:
    zahl = int(input("Zahl eingeben:"))
    print(10 / zahl)
except ValueError:
    print("keine Zahl")
except ZeroDivisionError:
    print("Division durch 0 falsch")
```

Das ist Fehlerbehandlung: Der try-Block wird versucht auszuführen, und je nachdem, welcher Fehler auftritt, greift der passende except-Zweig. Mir gefällt, dass man so gezielt auf unterschiedliche Fehlerarten reagieren kann (falsche Eingabe vs. Division durch 0), statt dass das Programm einfach abstürzt.

9. Bedingte Ausdrücke (Kurzform von if/else)

```
# ohne bedingte Ausdrücke
if alter >= 18:
    status = "volljährig"
else:
    status = "minderjährig"
```

```
# mit bedingten Ausdrücken
alter = 20
status = "volljährig" if alter >= 18 else "minderjährig"
```

Das ist im Grunde ein normales if/else, nur in eine Zeile gepackt. Für kurze, einfache Zuweisungen finde ich das übersichtlicher als vier Zeilen if/else zu schreiben – bei komplexeren Bedingungen würde ich aber die lange Form bevorzugen, weil es sonst schnell unleserlich wird.

10. match-case

```
tag = 2

match tag:
    case 1:
        print("Montag")
    case 2:
        print("Dienstag")
    case 3:
        print("Mittwoch")
    case _:
        print("Unbekannter Tag")
```

match-case ist Pythons Variante eines switch-statements (seit Python 3.10). Der Unterstrich `_` am Ende fängt alle Fälle ab, die vorher nicht getroffen wurden – ähnlich wie else bei if-Ketten. Für viele feste Werte finde ich das lesbarer als eine lange elif-Kette.